

# EDUHACKERS

## Developer Build & Technical Research Dossier

Architecture, workflow, tech-stack options, APIs, wireframes, GitHub operating model and MVP build plan for the Eduvos marketplace concept.

**Status: research dossier - not a final implementation decision. All institutional, legal, payment and identity assumptions require validation before build.**

|                  |                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------|
| Prepared for     | EduHackers team, Hack Jam judges, Eduvos students                                                    |
| Primary owner    | Paballo Kali - Developer / Research Lead                                                             |
| Scope            | All Eduvos campuses; web app + mobile app + installable desktop experience                           |
| Baseline concept | Verified student marketplace + protected transactions + demand intelligence + Incubation Hub support |
| Version          | 20 September 2026                                                                                    |

# Interactive contents

Use this dossier as a build-research blueprint. It is meant to guide team decisions, not to lock the implementation before user and institutional validation.

1. Executive technical summary
2. Build principles and research status
3. Product modes to support
4. Recommended MVP technology approach
5. Stack options and rationale
6. Reference architecture
7. Repo structure and GitHub setup
8. Branching, PR and CI workflow
9. Environment and deployment model
10. Domain model and ERD
11. RBAC and permissions
12. API strategy and endpoint research
13. OpenAPI and response contracts
14. Wireframe research - buyer flows
15. Wireframe research - seller flows
16. Protected payments and order states
17. Delivery, pickup and rentals
18. AI compliance and moderation
19. Search and Demand Intelligence
20. Storage, media and evidence
21. Testing strategy
22. Security and privacy architecture
23. Developer sprint workflow
24. MVP build roadmap
25. Risk register and open research questions
26. Appendices: repo skeleton, issue templates, endpoints, references

# 1. Executive technical summary

**Core recommendation: build a web/PWA MVP first, isolate payments and AI behind interfaces, and instrument every key marketplace event from the start.**

The EduHackers build should be researched as a **modular marketplace platform**, not as a simple listings application. The product must coordinate identity, listings, protected payment states, delivery/pickup, messaging, disputes, trust progression, events, gigs, rentals, digital products, AI moderation and Incubation Hub intelligence.

The recommended technical posture is **boring and reliable for the MVP**: a web-first installable PWA, a relational database, typed APIs, strong role-based access control, a protected GitHub workflow and carefully isolated payment/AI integrations. A native mobile app can be researched in parallel but should not block the MVP unless Eduvos requires app-store distribution.

The biggest build risk is not rendering screens. The hard parts are money movement, object-level authorization, data privacy, dispute evidence, file security, moderation, marketplace liquidity and institutional governance. The architecture must therefore be designed around boundaries, auditability and change.

Stack decision logic: use boring reliable tech for MVP, isolate risky integrations

Frontend

Next.js + PWA - Fast web/PWA, easy preview URLs

Database

Postgres - Strong relational model + analytics

Storage

Supabase / S3 style - Evidence files, images, digital products

AI

LLM moderation + rules - Human review for punishment

Native later

Expo - Only if app-store/native APIs matter

Backend

Next route handlers / server functions - Simpler modular monolith

Payments

PSP marketplace flow - Do not hold money directly

Testing

Vitest + Playwright - unit, UI, E2E gates

## 2. Research status: what is known vs still hypothetical

This document builds on the prior EduHackers marketplace research dossier. Several product choices are user-selected hypotheses: integrated payments, held funds, 2% commission, campus-first discovery, student couriers, pickup points, rentals, services with 30% deposits, AI compliance, Trusted Seller progression and Incubation Hub 30-day intervention plans.

None of these are final implementation facts yet. The developer research must distinguish **user experience intent** from **technical/legal feasibility**. For example, the app can show money as “held”, but actual held funds should be managed by a regulated payment service provider or banking partner rather than by student-written code.

The research must also avoid turning the system into a super-app too early. The MVP should prove the most valuable loop: verified users, campus discovery, protected order state, fallback/request capture, demand signal generation and Incubation Hub actionability.

| Area                       | Current research confidence              | Developer implication                                    |
|----------------------------|------------------------------------------|----------------------------------------------------------|
| Listings, profiles, search | High baseline feasibility                | Straightforward MVP domain work                          |
| Protected payments         | Medium feasibility; high compliance risk | Use PSP references, not internal money ledger only       |
| Student delivery network   | Operationally uncertain                  | Prototype status flow before routing/live tracking       |
| AI moderation              | Useful but error-prone                   | AI assists; human review for high-impact action          |
| Incubation intelligence    | Strong differentiation hypothesis        | Needs careful privacy boundaries and explainable signals |
| Campus reels/social feed   | Valuable but moderation-heavy            | Phase after core commerce proof                          |

### 3. Product modes the build must research

The marketplace is not one workflow. It contains different commercial modes that share identity, trust, messaging, payments, dispute handling and analytics, but each has separate state rules. Treating all listings as one generic object will create technical debt.

The research architecture should define a shared Listing core with mode-specific extensions. Product listings need stock and condition. Services need availability, booking and 30% deposit logic. Rentals need deposits, return deadlines and condition evidence. Digital products need licensing, download logs and file integrity. Food listings need ingredients, allergens and category safety checks.

| Mode            | Core fields                                  | Special workflow                                        |
|-----------------|----------------------------------------------|---------------------------------------------------------|
| Product         | title, price, campus, stock, pickup/delivery | stock reservation, confirmation, return/refund          |
| Service         | brief, duration, availability, deposit       | booking, 30% deposit, evidence, milestone/completion    |
| Rental          | item, value, deposit, due date               | condition evidence, late fees, return dispute           |
| Digital product | file, license, version, category             | protected payout, access log, copyright/academic checks |
| Food            | ingredients, allergens, prep time, storage   | food safety gate, perishable delivery timing            |
| Event/gig       | organiser, capacity, tickets/applications    | RSVP, ticket, waitlist, vendor applications             |

## 4. Delivery surfaces: web, mobile and desktop install

The preferred research direction is **PWA-first**: build a responsive web app that can be installed on desktop and mobile. This directly supports the user requirement for web + mobile + downloadable desktop experience while keeping one codebase for the MVP.

MDN's PWA guidance confirms that installable PWAs use a web app manifest and HTTPS; once installed, they can appear and launch like a platform-specific app. Browser support varies across desktop and mobile platforms, so the user experience must gracefully fall back to normal web access where installation is limited. [D1]

A native mobile app remains a serious phase-two option if camera scanning, push notifications, app-store discoverability, reliable background location for couriers or native performance become decisive. Expo/EAS can produce Android and iOS binaries from React Native projects and support internal distribution builds. [D2]

| Surface              | Recommended role                  | Research decision                                |
|----------------------|-----------------------------------|--------------------------------------------------|
| Responsive web app   | Primary MVP surface               | Fastest build and easiest team preview           |
| Installable PWA      | Desktop/mobile install experience | Use manifest, icons, service worker where useful |
| Expo mobile app      | Phase-two native shell            | Use if app-store/native APIs become necessary    |
| Desktop packaged app | Future option only                | PWA desktop install likely enough for Hack Jam   |

## 5. Recommended tech-stack direction

The build should optimise for speed, developer control, security and visibility during the Hack Jam. A TypeScript stack is appropriate because it can share types between frontend, API contracts and validation. Next.js App Router route handlers can create custom request handlers inside the app directory using standard Web Request/Response APIs. [D3]

Supabase is a strong research candidate because it provides Postgres, Auth, Storage, Realtime, Edge Functions, APIs and Row Level Security in one platform. Supabase documentation states that every project includes a Postgres database with Realtime capabilities, Auth, Storage and Edge Functions. [D4]

Postgres is the recommended data model foundation because the marketplace is relational: users, roles, listings, orders, payments, disputes, evidence, events and Incubation records need strong consistency and queryability.

| Layer    | Recommendation                                                    | Why                                                    |
|----------|-------------------------------------------------------------------|--------------------------------------------------------|
| Frontend | Next.js + TypeScript + Tailwind/shadcn-style components           | Web-first, type-safe, fast previews, mobile responsive |
| Backend  | Next.js route handlers / server actions + service layer           | Simple modular monolith, fewer moving parts            |
| Database | PostgreSQL via Supabase or equivalent                             | Relational consistency and analytics                   |
| Auth     | Supabase Auth or Clerk/Auth.js depending Eduvos verification path | Avoid custom auth unless required                      |
| Files    | Supabase Storage / S3-style storage                               | Images, dispute evidence, digital products             |
| Realtime | Supabase Realtime or websocket provider                           | Messages/order updates where needed                    |
| Payments | PSP marketplace capabilities                                      | Do not self-hold funds                                 |
| Testing  | Vitest + Playwright + Storybook                                   | Unit, E2E and UI validation                            |

## 6. Tech-stack alternatives to research before final choice

The recommended stack is not the only possible stack. The team should evaluate alternatives on team skill, cost, compliance, payment support, identity integration, deployment speed and how easily the Hack Jam prototype can demonstrate the full loop.

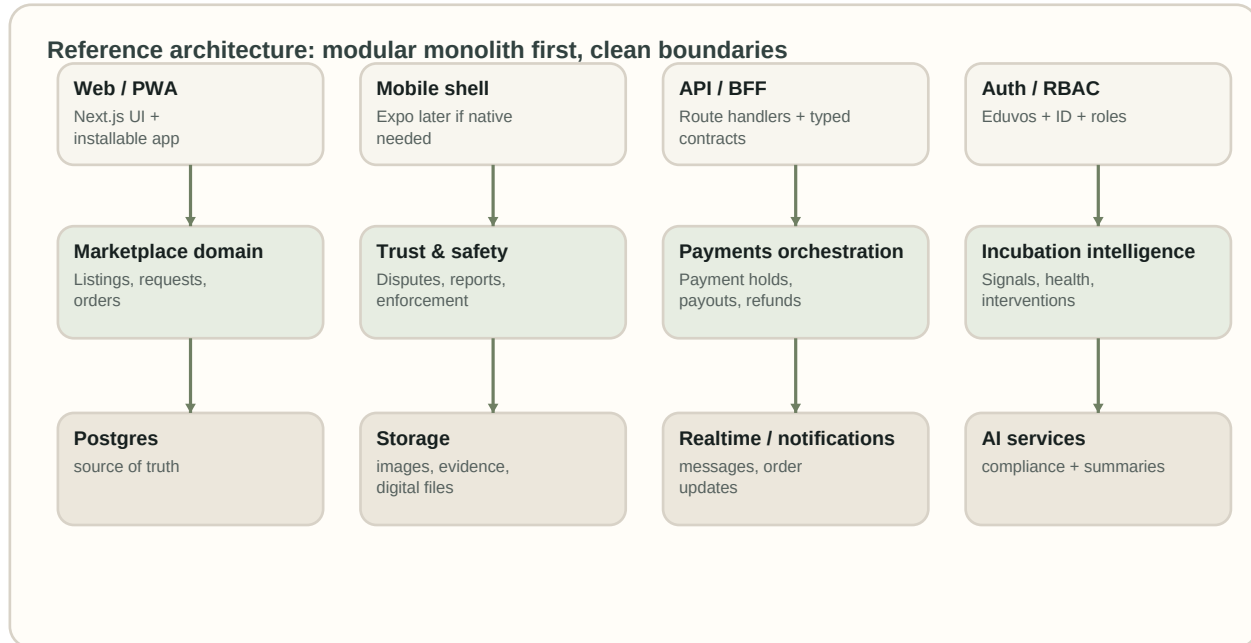
Firebase can move quickly but may become awkward for relational transactions and complex reporting. Supabase/Postgres fits relational marketplace data better. A fully custom Node/Nest backend gives maximum control but costs more time. A no-code/low-code backend can prototype quickly but may be weaker for custom payment/dispute logic and security depth.

| Option                           | Strength                       | Concern                                       | Recommendation                    |
|----------------------------------|--------------------------------|-----------------------------------------------|-----------------------------------|
| Next.js + Supabase               | Fast, relational, PWA-friendly | RLS and policies must be carefully tested     | Best MVP research candidate       |
| Next.js + Prisma + Neon/Postgres | Strong typed data layer        | More auth/storage/realtime pieces to assemble | Strong alternative                |
| NestJS API + React web           | Clean backend architecture     | Slower setup for small team                   | Research only if complexity grows |
| Firebase                         | Quick auth/realtime            | Relational reporting and payment holds harder | Not preferred for this domain     |
| Flutter/React Native first       | Native mobile UX               | Slower web/desktop and team preview           | Phase-two candidate               |

## 7. Reference architecture

The architecture should begin as a modular monolith: one deployment with clearly separated modules. This avoids premature microservices while preserving boundaries. The payment, AI, notification and identity integrations should be replaceable adapters, not hard-coded inside UI components.

The system should have separate bounded areas for identity, marketplace catalogue, orders/bookings/rentals, payments, delivery/pickup, trust/safety, social discovery, analytics/demand intelligence and Incubation Hub support.



## 8. GitHub repository structure

The repository should be private during the Hack Jam unless Eduvos explicitly instructs otherwise. It should contain code, documentation, architecture decisions, API contracts, design handoff references, tests and deployment configuration.

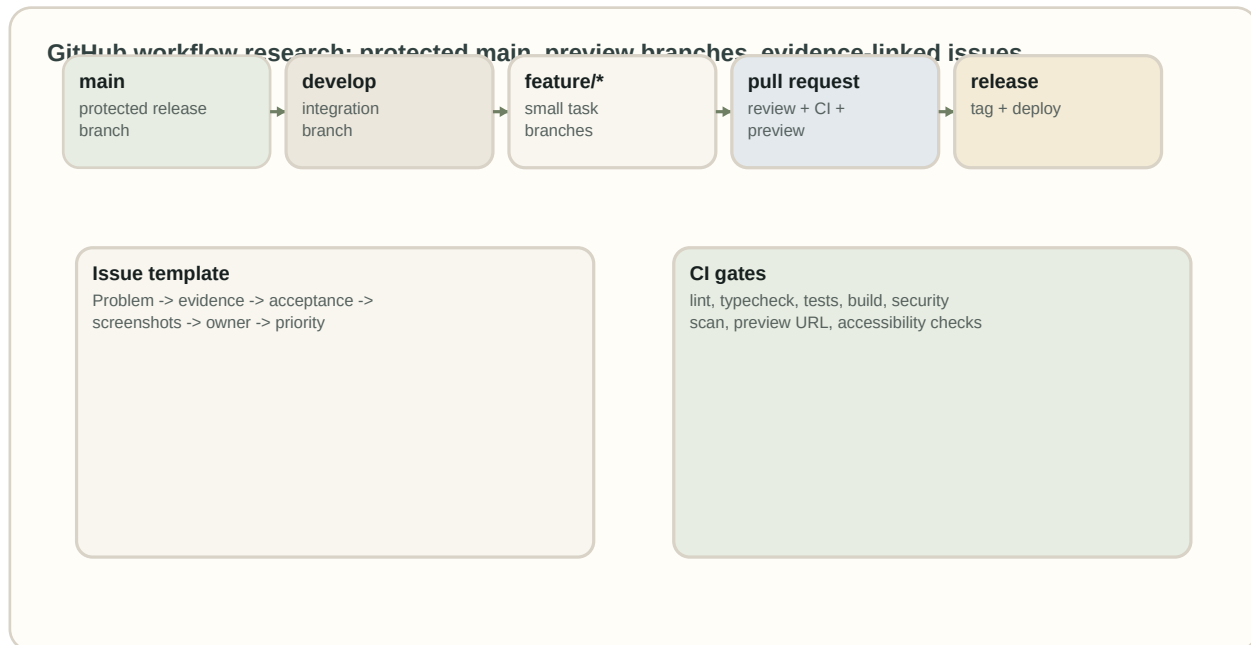
A monorepo is useful if the web app, shared types, database schema and documentation need to evolve together. Do not over-engineer packages until the team has a working MVP. Start clean, with explicit folders and issue templates.

| Path                   | Purpose                                                   |
|------------------------|-----------------------------------------------------------|
| apps/web               | Next.js marketplace web/PWA app                           |
| packages/ui            | Shared UI primitives, design tokens, form components      |
| packages/shared        | Shared TypeScript types, enums, validators                |
| packages/db            | Prisma/Supabase schema, migrations, seed data             |
| docs/research          | Research notes, assumptions, links to PDF evidence        |
| docs/architecture      | ADRs, ERD, RBAC, API contracts                            |
| docs/security          | Threat model, ASVS checklist, privacy model               |
| .github/workflows      | CI, tests, build, security checks                         |
| .github/ISSUE_TEMPLATE | Bug, feature, research evidence, design handoff templates |
| tests/e2e              | Playwright task-based user journeys                       |

## 9. GitHub operating model

GitHub branch protection should be used so the main branch cannot be casually overwritten. GitHub protected branches can require pull request reviews, required status checks, conversation resolution, signed commits, merge queues and push restrictions. [D5]

The workflow should force traceability: every feature starts from research evidence or a product decision, becomes an issue, gets a branch, receives a pull request, passes CI, receives review and is tested on a preview deployment before it is considered done.



# 10. Branching and pull request rules

The Hack Jam team does not need a complex enterprise branching strategy. Use a lightweight trunk-style approach with protected main, a develop branch if the team wants staging, and short-lived feature branches. All code changes should go through pull requests.

The developer owner should not allow direct main pushes. This prevents last-minute breakage and gives UX/Product/Research space to review whether the implementation still matches the evidence and design.

| Branch    | Rule                                   | Purpose                             |
|-----------|----------------------------------------|-------------------------------------|
| main      | protected, production/demo-ready only  | Stable demo and release branch      |
| develop   | integration branch, optional protected | Shared QA before release            |
| feature/- | one task per branch                    | Small changes easier to review      |
| fix/-     | bugs from testing                      | Keeps defect work visible           |
| docs/     | research/architecture updates          | Treat documentation as product work |

# 11. CI/CD research and deployment environments

GitHub Actions workflows are YAML files stored under `.github/workflows` and define automated jobs triggered by repository events. [D6] The minimum CI should run linting, type checks, unit tests, build, database migration validation and E2E tests for critical flows.

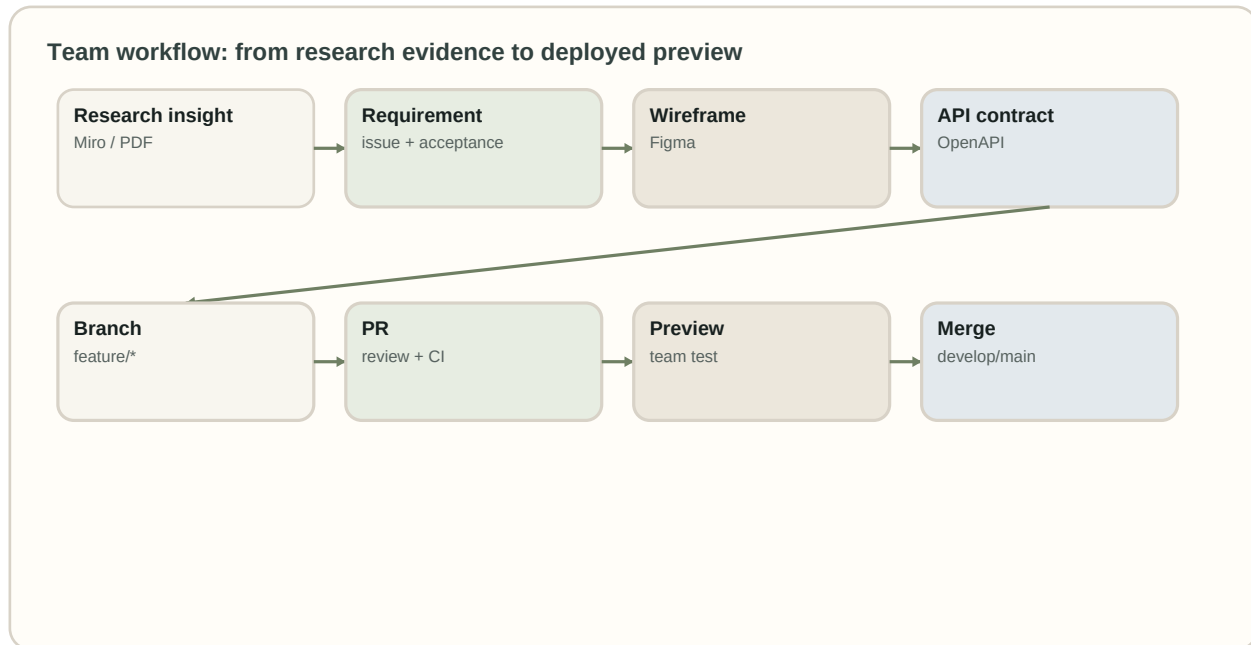
Vercel provides Local, Preview and Production environments. Preview deployments can be created for pull requests or non-production branches, which makes them useful for team QA and stakeholder review. [D7] Environment variables on Vercel are configured outside source code and encrypted at rest, but changes apply only to new deployments. [D8]

| Environment     | Use                   | Rules                                             |
|-----------------|-----------------------|---------------------------------------------------|
| Local           | developer machine     | sample data; no production secrets                |
| Preview         | every PR / branch     | share URL for UX, Product and QA review           |
| Staging         | optional if supported | near-production data shape, fake payment provider |
| Production/Demo | final Hack Jam demo   | frozen seed data + backup demo path               |

## 12. Developer workflow from research to code

The workflow must keep research, product, UX and development connected. A developer should not build from a vague idea. Each task needs evidence, a product decision, UX acceptance, API/data impact and test expectation.

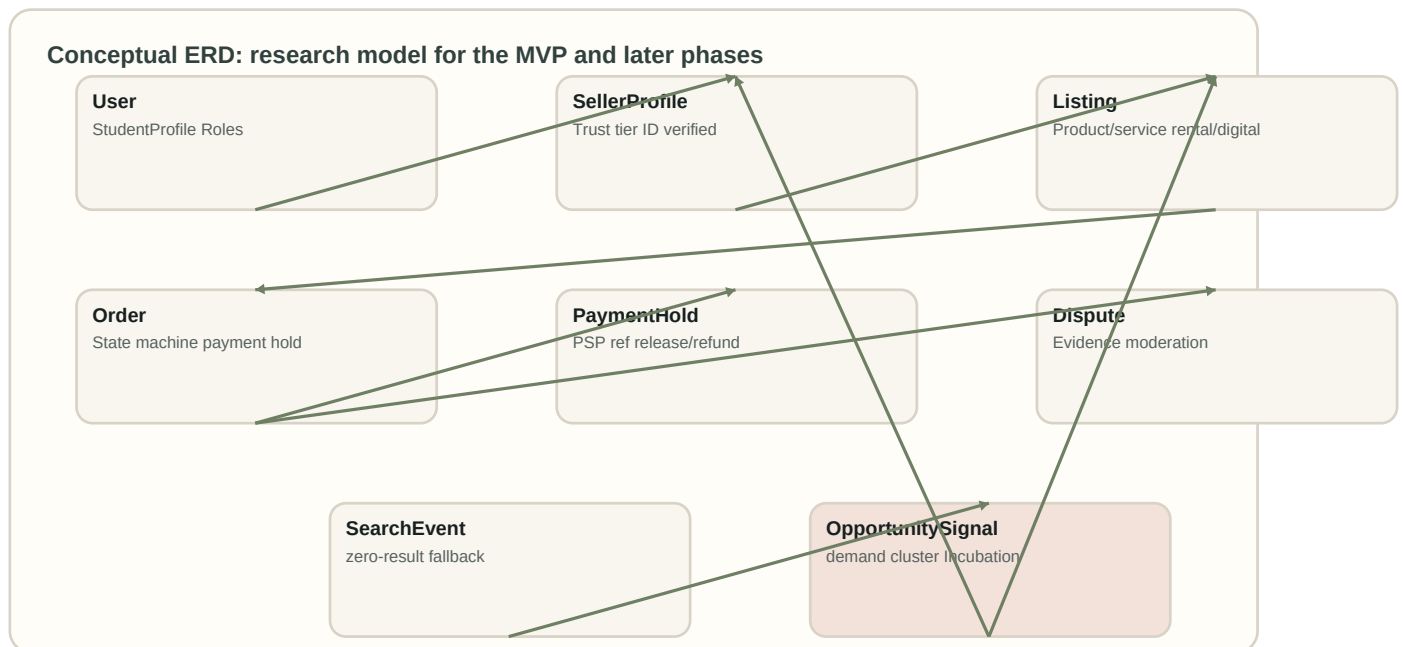
This prevents the team from building a huge set of disconnected features. It also makes the final presentation stronger: the team can show exactly why the feature exists and how it was validated.



## 13. Conceptual data model

The data model should start from marketplace modes and trust boundaries. The central objects are User, StudentProfile, SellerProfile, Listing, Availability, Order/Booking, PaymentHold, DeliveryTask, Review, Dispute, EvidenceFile, SearchEvent, OpportunitySignal, IncubationRecord and AuditLog.

The core differentiator is the relationship from SearchEvent to OpportunitySignal. It is not enough to log analytics for dashboards. The system must be able to distinguish fulfilled search, local failure, fallback success, request creation and repeated unmet demand.



## 14. RBAC and object-level authorization

OWASP API Security Top 10 2023 identifies Broken Object Level Authorization as the first API risk and emphasizes authorization checks for functions accessing data using user-supplied object IDs. [D9] This is directly relevant: a student should never be able to modify another seller's listing, payout state or dispute evidence by changing an ID in an API request.

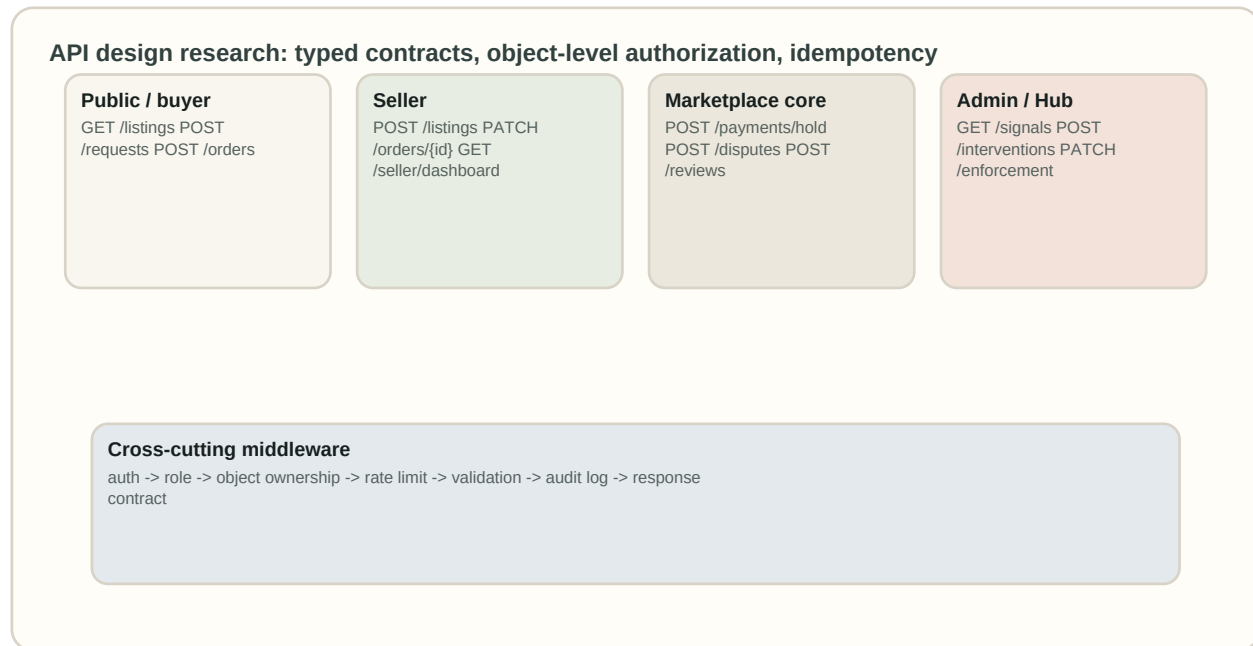
Use both role-based access control and object ownership checks. Roles alone are not enough. A user may be a seller, but they should still only edit their own seller profile, listings, order updates and evidence.

| Capability                 | Buyer | Personal Seller | Trusted Seller | Courier | Incubation Staff | Moderator     |
|----------------------------|-------|-----------------|----------------|---------|------------------|---------------|
| Buy/book/rent              | Yes   | Yes             | Yes            | Yes     | No               | No            |
| Create standard listings   | No    | Yes             | Yes            | No      | No               | No            |
| Business storefront        | No    | No              | Yes            | No      | No               | No            |
| Subscriptions              | No    | No              | Trusted only   | No      | No               | No            |
| Accept delivery tasks      | No    | Optional        | Optional       | Yes     | No               | No            |
| View Business Health Score | No    | No              | No             | No      | Authorized only  | Limited/admin |
| Resolve disputes           | No    | No              | No             | No      | No               | Yes           |
| Apply enforcement          | No    | No              | No             | No      | No               | Yes           |

# 15. API strategy: route handlers, contracts and validation

Use a Backend-for-Frontend style API inside the application during the MVP. Next.js route handlers can implement GET, POST, PUT, PATCH, DELETE and other HTTP methods in app/api route files. [D3]

Each endpoint should validate inputs server-side, enforce auth and object ownership, return typed error codes, and write audit logs for sensitive flows. Public client state should not be trusted for price, commission, trust status, payout eligibility or authorization.



## 16. OpenAPI and endpoint research

OpenAPI defines a language-agnostic interface description for HTTP APIs so humans and computers can understand service capabilities without reading source code. [D10] The team should create an OpenAPI-style contract even if the MVP is implemented inside Next.js, because it clarifies frontend-backend responsibilities.

The endpoint table below is a research contract. It is not final. It provides enough structure for Figma screens, API mocks, GitHub issues and tests.

| Domain        | Example endpoints                                  | Notes                                       |
|---------------|----------------------------------------------------|---------------------------------------------|
| Auth/identity | POST /auth/verify-student, POST /identity/id-check | May depend on Eduvos and provider decisions |
| Marketplace   | GET /listings, POST /listings, GET /listings/{id}  | Mode-specific validation                    |
| Availability  | GET /availability, POST /availability/slots        | Stock and service slots differ              |
| Requests      | POST /requests, GET /requests/matches              | Zero-result and custom brief flow           |
| Orders        | POST /orders, PATCH /orders/{id}/state             | Idempotency key required                    |
| Payments      | POST /payments/hold, POST /payments/release        | PSP references only                         |
| Disputes      | POST /disputes, POST /disputes/{id}/evidence       | Evidence access controls                    |
| AI compliance | POST /ai/check-listing, POST /ai/summarize-signal  | Human review path                           |
| Incubation    | GET /incubation/signals, POST /interventions       | Opt-in and staff roles                      |

# 17. API response conventions and error handling

API consistency matters because the project has many roles and modes. The frontend should not need to guess what a failure means. Use stable response envelopes, typed error codes and field-level validation messages.

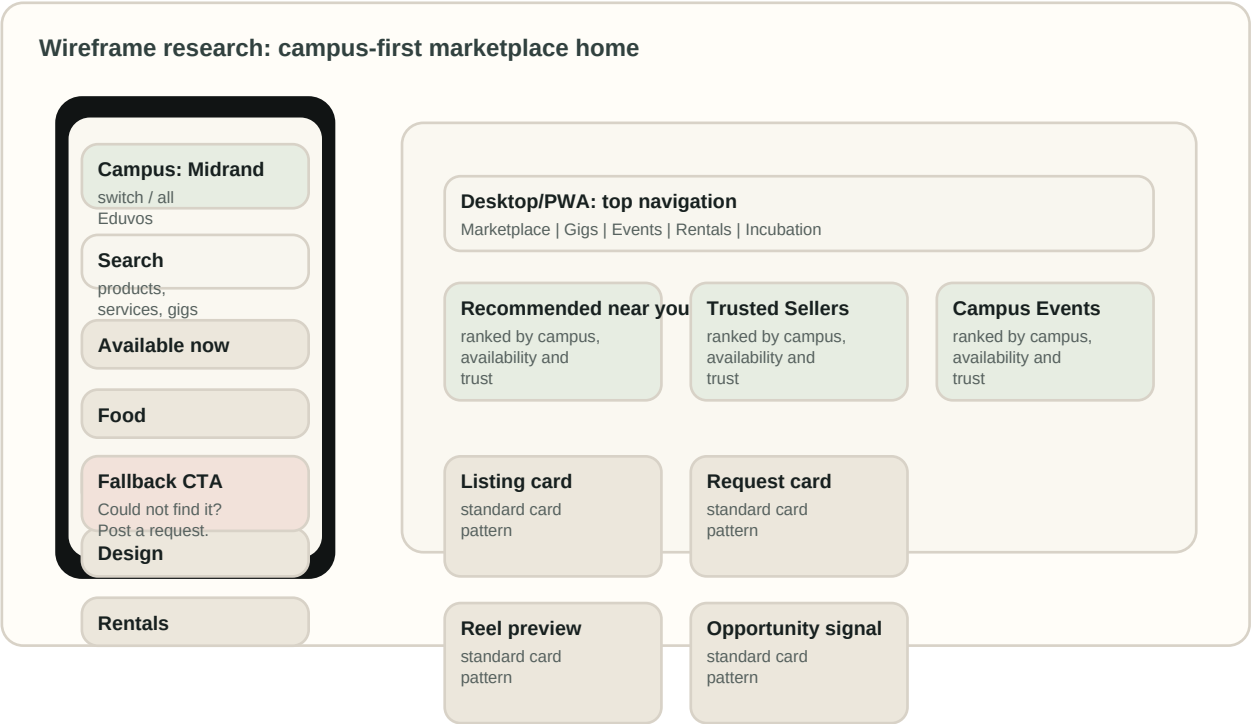
For sensitive flows, idempotency keys should prevent duplicate orders or duplicate payment-hold attempts if the user taps twice or the network retries. Audit logs should record who initiated the action, what object was affected, and why.

| Pattern           | Research recommendation                                              |
|-------------------|----------------------------------------------------------------------|
| Success envelope  | { ok: true, data, meta }                                             |
| Error envelope    | { ok: false, error: { code, message, fields?, traceId } }            |
| Validation        | Use schema validation for every POST/PATCH                           |
| Authorization     | check role + object ownership + state transition                     |
| State transitions | reject invalid transitions with typed error                          |
| Idempotency       | required for payment, order, ticket, rental and subscription actions |
| Audit trail       | mandatory for moderation, disputes, payments and identity            |

# 18. Wireframe research: buyer discovery and fallback

The buyer flow should not end at “no results”. The UI must support local campus search first, then availability, similar sellers, other campuses, alternative dates and finally a request. This is the UX expression of Demand Intelligence.

The wireframes should test whether users understand the fallback sequence. If students feel like cross-campus results are irrelevant or confusing, the product must make the scope explicit: “Available at another Eduvos campus”, “Can ship to your campus”, or “Post a request locally”.



# 19. Wireframe research: seller onboarding and listing creation

Seller onboarding must separate eligibility from public identity. The platform requires Eduvos student verification and ID verification, but the public seller profile can begin as a personal verified profile. Brand storefront unlock comes later after 10 completed sales, recommended R1,000 completed GMV and good standing.

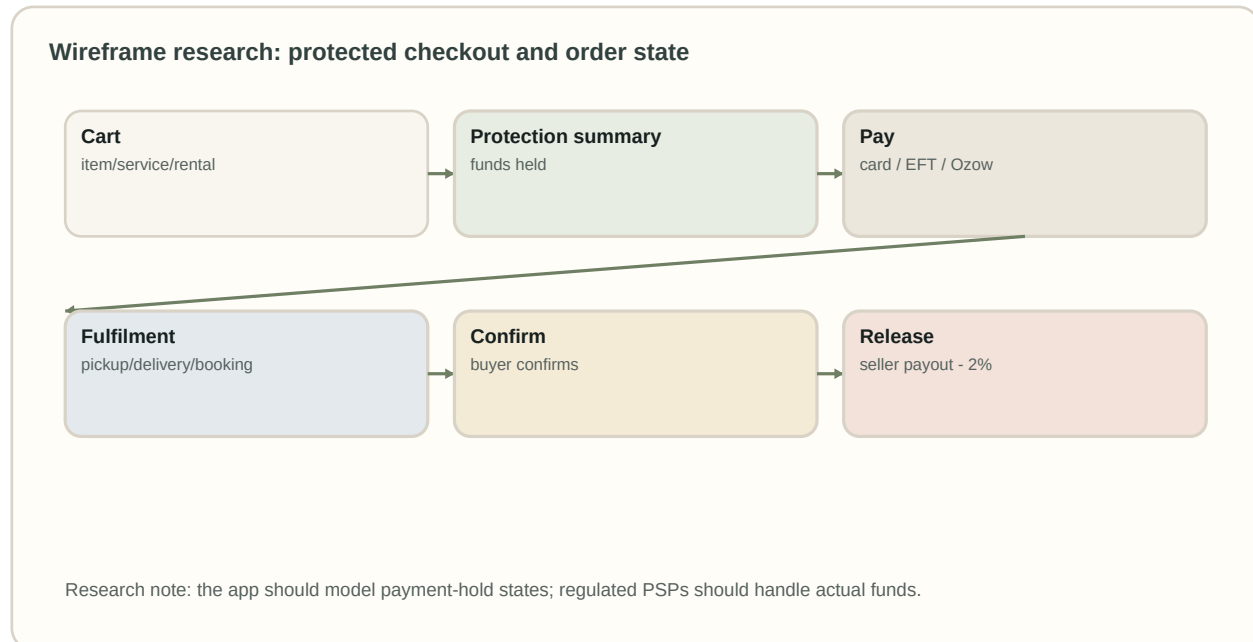
Listing creation should be mode-aware. A food listing must ask for ingredients/allergens. A rental listing must ask for deposit and condition evidence. A service listing must ask for availability, 30% deposit behavior and scope. A digital listing must ask for license and file ownership.

| Onboarding step         | Data needed                            | Build note                                |
|-------------------------|----------------------------------------|-------------------------------------------|
| Create account          | email/phone, password/passkey if used  | use provider auth                         |
| Eduvos verification     | student email/number verification path | do not expose raw student number publicly |
| ID verification         | legal name, ID result/status           | store status separately from storefront   |
| Personal seller profile | display name, campus, categories       | first selling stage                       |
| Trust progression       | sales count, GMV, rating, disputes     | computed server-side                      |
| Brand storefront unlock | business name, logo, story             | only after trust threshold                |

## 20. Wireframe research: checkout and payment hold

The checkout flow should show payment protection in plain language. Users must understand when funds are charged, where the 2% commission appears, when the seller is paid, and how disputes/refunds work.

The app can display a payment protection timeline even if the underlying provider implementation is still stubbed in the MVP. This lets the team test comprehension before live money movement.



## 21. Payment architecture research

The app should not directly hold student money. The internal domain model should represent a PaymentHold state, while a regulated payment provider or banking partner manages authorisation, settlement, refunds and payouts. The prior marketplace research already found that payment-processing fees may exceed the proposed 2% commission on some rails.

Payment research must determine the provider's support for subaccounts, split payments, delayed release, partial refunds, chargebacks, payout KYC, refund workflows and fee allocation. Crypto should remain later-phase research due to volatility, compliance, refunds and disputes.

| Payment method            | MVP research status       | Risk                                             |
|---------------------------|---------------------------|--------------------------------------------------|
| Card                      | common and convenient     | processor fees can exceed 2% commission          |
| Instant EFT / pay-by-bank | promising lower-cost rail | user trust and coverage need testing             |
| Normal EFT                | possible manual fallback  | slow confirmation and reconciliation burden      |
| Wallets/mobile payments   | phase research            | provider availability varies                     |
| Crypto                    | later phase only          | volatile, refunds/disputes/compliance complexity |

## 22. Delivery, pickup and courier workflow

The fulfilment system should support campus pickup, seller delivery and verified student couriers. For cross-campus items, research includes student courier transfer, seller shipping to buyer campus and buyer collection from the other campus.

Designated pickup points reduce risk and create a clear proof-of-handoff model. The build should model pickup zones as first-class objects with campus, operating hours, allowed modes and handoff verification method.

| Fulfilment mode            | Key data                                    | Research issue                    |
|----------------------------|---------------------------------------------|-----------------------------------|
| Campus pickup              | pickup point, time window, handoff code     | safety and operating hours        |
| Seller delivery            | route, fee, ETA, evidence                   | seller availability and liability |
| Student courier            | courier eligibility, task acceptance, proof | safety, insurance, reliability    |
| Cross-campus ship          | carrier/transfer ref, ETA, fee              | cost and delivery time            |
| Buyer collection elsewhere | campus, time, access rules                  | practicality and safety           |

## 23. Rentals, second-hand and digital products

These modes should be researched as dedicated flows, not bolted on later. Rentals need refundable deposits, condition evidence, return deadlines and automatic late charges. High-value rentals need stronger verification and possibly stricter renter eligibility.

Second-hand products need condition grading and defect disclosure. Digital products need file ownership, versioning, download logs, licensing and anti-abuse controls because a buyer can download first and dispute later.

| Mode              | Special controls                                     | MVP stance                          |
|-------------------|------------------------------------------------------|-------------------------------------|
| Rental            | deposit, condition photos/video, due time, late fees | prototype one category only         |
| High-value rental | higher deposit, extra verification, evidence         | phase carefully                     |
| Second-hand       | condition grade, defect disclosure, photos           | MVP-friendly                        |
| Digital product   | file hash, version, license, access logs             | prototype with simple download gate |
| Subscription      | Trusted Seller only, category-limited                | defer until one-off model validated |

## 24. AI compliance and moderation architecture

AI should be visible as a compliance and marketplace-quality assistant. Its job is to reduce risk and guide users, not to silently make high-impact decisions. AI can pre-check listings, flag prohibited items, detect missing food allergens, cluster demand, summarize seller analytics and propose Incubation interventions.

The product policy should separate content state from account punishment. AI can block or hide a listing for review where policy match is clear. Warnings, probation, suspension and dispute outcomes need a human review or appeal path.

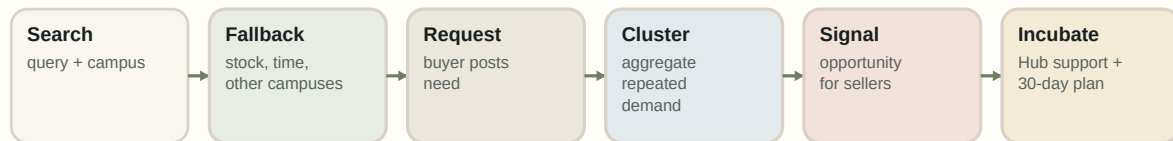
| AI use case        | Input                                | Output                    | Human control                  |
|--------------------|--------------------------------------|---------------------------|--------------------------------|
| Listing compliance | title, description, category, images | allow/flag/block + reason | moderator review for penalties |
| Food safety check  | ingredients/allergens/storage        | missing-field warnings    | not a hygiene certification    |
| Demand clustering  | searches/requests                    | opportunity signal        | Research/Hub interprets        |
| Seller coaching    | order metrics                        | specific recommendation   | seller sees guidance only      |
| Hub intervention   | health indicators                    | 30-day plan suggestion    | staff approves/edits           |

## 25. Demand Intelligence system design

Demand Intelligence is a data product. It needs event instrumentation, aggregation thresholds, privacy controls and explainable outputs. The system should not turn one search into a claimed market opportunity. It should look for repeated patterns by campus/category/time and show confidence levels.

Repeated unmet demand can lead to three actions: show existing sellers an extension opportunity, show aspiring entrepreneurs a tested idea to explore, and show the Incubation Hub a support or supply gap.

### Demand Intelligence pipeline: capture lost demand before it leaks



Core research risk: this must be validated with Eduvos users before being treated as product truth.

## 26. Storage and evidence-file architecture

The platform will store profile images, listing images, food photos, delivery proof, dispute evidence, digital downloads, rental condition evidence and possibly short video reels. Storage access must be role-aware. Supabase Storage supports access control with Postgres Row Level Security policies on storage objects. [D11]

Evidence files should not be treated like public listing images. They require access controls, retention rules, audit logs and possibly immutable references. Digital products require access logs, file versioning and hash records.

| Bucket/type      | Access                                         | Retention research             |
|------------------|------------------------------------------------|--------------------------------|
| listing-images   | public/authenticated read depending policy     | while listing active + archive |
| seller-documents | private; staff/provider only                   | legal/institutional policy     |
| dispute-evidence | buyer/seller/moderator scoped                  | retain through appeal window   |
| digital-products | buyer access after payment; seller owns upload | versioned + logs               |
| rental-condition | transaction scoped                             | retain through deposit window  |
| reels/videos     | public campus feed if approved                 | moderation + takedown rules    |

## 27. Realtime messaging and notifications

The platform needs structured order notifications and free-form in-app messaging. The safest design is not to depend on chat for the truth of the transaction. Listing terms, order state, booking slot, deposit, delivery method and dispute evidence should be stored as structured data.

Supabase Realtime supports listening to database changes; its documentation recommends Broadcast for scalability and security in many cases. [D12] For MVP, realtime can be limited to order updates, chat threads and moderation status changes.

| Notification type      | Trigger                   | Channel                      |
|------------------------|---------------------------|------------------------------|
| Order accepted         | seller changes state      | in-app + optional email/push |
| Payment protected      | PSP webhook / stub        | in-app receipt               |
| Ready for pickup       | seller/delivery update    | in-app + reminder            |
| Dispute opened         | buyer/seller action       | in-app + moderator queue     |
| Opportunity signal     | cluster threshold reached | seller/Hub dashboard         |
| 30-day plan checkpoint | intervention schedule     | seller + staff dashboard     |

## 28. Testing strategy: unit, component, E2E and security

Testing must focus on business-critical journeys: identity, seller onboarding, listing creation, checkout, order state, payment hold, delivery, dispute, moderation, demand signal and role restrictions.

Playwright provides reliable end-to-end automation for modern web apps across Chromium, Firefox and WebKit, with auto-waiting, assertions, tracing and parallelism. [D13] Storybook supports component testing and visual testing patterns; stories can become test cases for UI states. [D14]

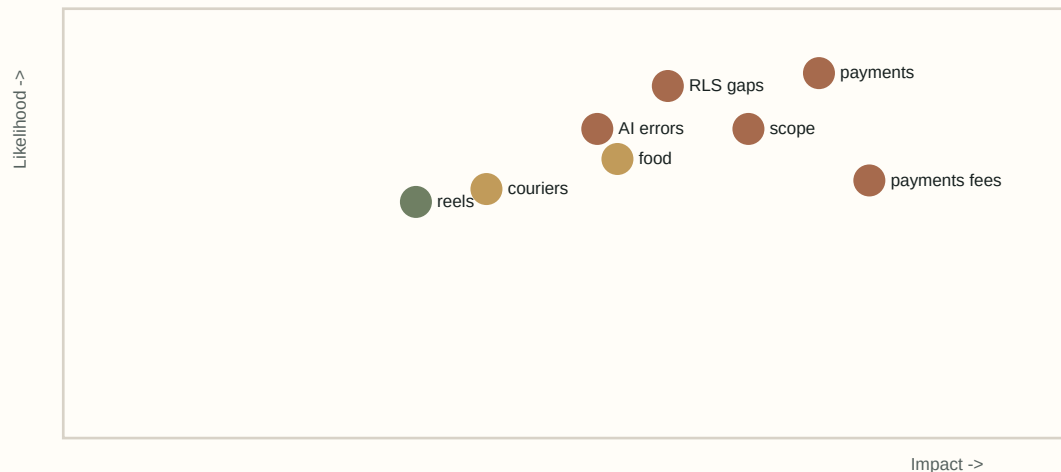
| Layer         | Tool candidate                      | What to test                                            |
|---------------|-------------------------------------|---------------------------------------------------------|
| Unit          | Vitest                              | validators, pricing, commission, state machines         |
| Component     | Storybook/Vitest                    | cards, forms, checkout states, error states             |
| E2E           | Playwright                          | buyer purchase, seller fulfilment, dispute, role checks |
| Security      | OWASP/ASVS checklist + manual tests | BOLA, auth, file upload, rate limits                    |
| Accessibility | axe/Storybook/Playwright checks     | labels, focus, contrast, keyboard flows                 |
| Performance   | Lighthouse / Web Vitals             | mobile response and bundle size                         |

## 29. Security architecture and threat model

The system must be designed as a security-sensitive marketplace. Students can lose money, reputations, evidence files and personal information if controls fail. OWASP API Security states APIs expose business logic and sensitive data and are major attacker targets. [D15]

The highest-risk abuse paths are: broken object authorization, fake sellers, account takeover, manipulated payment states, fraudulent disputes, evidence tampering, malicious uploads, courier impersonation, academic-misconduct listings and AI moderation errors.

**Build-risk heat map: research priorities before implementation**



## 30. Privacy, POPIA and data minimization

The product should know less than it technically could. Identity verification data, student verification, marketplace profiles, analytics and Incubation Hub intelligence must be separated. The public profile should never expose raw student numbers or government-ID details.

Precise location should be optional and task-specific. Campus is the default context. Courier live-location history should have strict retention. Business Health Scores should be internal to authorised Incubation staff while sellers receive actionable guidance, not raw internal scoring.

| Data class                  | Who should see it                        | Research control                          |
|-----------------------------|------------------------------------------|-------------------------------------------|
| Student verification status | system, limited staff                    | status only in app                        |
| Government ID data          | verification provider / authorized staff | do not expose publicly                    |
| Seller storefront           | buyers                                   | public-safe brand/persona                 |
| Analytics events            | research/Hub aggregate                   | pseudonymous; aggregation thresholds      |
| Dispute evidence            | parties + moderator                      | role scoped, audit logged                 |
| Business Health Score       | Incubation staff                         | not seller-facing; explainable components |

## 31. Database and RLS design research

If Supabase is selected, Row Level Security policies become a major research and testing item. Supabase documentation explains that policies are attached to tables and run each time the table is accessed; they act like a WHERE clause for queries. [D16]

However, do not rely only on RLS. Server route handlers should still enforce authorization for sensitive state transitions, especially payments, disputes, moderation, identity, Trust Seller unlocks and Incubation interventions. Use defense in depth.

| Table              | RLS pattern to research                                   |
|--------------------|-----------------------------------------------------------|
| profiles           | user can read public fields; edit own profile only        |
| seller_profiles    | seller edits own; public reads approved storefront fields |
| listings           | seller owns writes; buyers read active/approved listings  |
| orders             | buyer/seller read only their transaction                  |
| payment_holds      | no direct client write; service role only                 |
| disputes/evidence  | participants + moderator scoped                           |
| incubation_records | staff only; seller opt-in controls outreach               |
| audit_logs         | append-only service role; no client read                  |

## 32. Developer setup and local environment

The team should document setup so any developer can run the project consistently. A broken setup wastes Hack Jam time. Use a predictable package manager, environment-variable template, seed data and local development commands.

Avoid putting secrets in the repository. Use .env.example for names only. Pull real secrets from Vercel/Supabase dashboard or CLI where permitted. Development should use fake payment provider endpoints until a payment partner is approved.

| Setup item        | Recommendation                                           |
|-------------------|----------------------------------------------------------|
| Node version      | Pin in .nvmrc or package manager config                  |
| Package manager   | pnpm or npm consistently; do not mix lockfiles           |
| Env files         | .env.example committed; .env.local ignored               |
| Database          | local Supabase/Postgres or hosted dev DB with seed data  |
| Seed users        | buyer, seller, trusted seller, courier, staff, moderator |
| Demo data         | listings, requests, orders, disputes, signals, events    |
| Fake integrations | payment, ID verification, email/SMS, AI responses        |

# 33. GitHub issue, PR and documentation templates

Templates reduce ambiguity. The team should not open tasks named “fix page” or “make checkout”. Each issue should declare the user role, evidence, requirement, acceptance criteria, designs and test notes.

Pull requests should explain what changed, what evidence or issue it implements, what tests passed, which screenshots changed, and what risks remain.

| Template       | Required fields                                                 |
|----------------|-----------------------------------------------------------------|
| Feature issue  | problem, evidence link, user story, acceptance, owner, priority |
| Bug issue      | steps, expected/actual, screenshot, severity, environment       |
| Research issue | question, source, finding, implication, confidence              |
| UX handoff     | Figma link, states, empty/error states, interactions            |
| PR template    | linked issue, summary, tests, screenshots, risk, rollback       |

## 34. Build workflow by role

The developer is not working alone. Paballo owns technical implementation and research traceability. Oratile owns UX/UI design handoff. Dikeledi owns business requirements and acceptance logic. Mpumelelo owns product prioritization and scope decisions. Danny owns timeline, blockers and pitch evidence.

The technical workflow should make these roles operational: Product approves what is in scope, UX provides states, Business provides rules, Developer implements, Project/Pitch captures proof, and Research verifies whether the result solves the original problem.

| Role                    | Technical handoff                                               |
|-------------------------|-----------------------------------------------------------------|
| Paballo - Dev/Research  | architecture, API, implementation, tests, research traceability |
| Oratile - UX/UI         | Figma flows, states, component specs, usability feedback        |
| Dikeledi - BA/Marketing | business rules, acceptance criteria, adoption metrics           |
| Mpumelelo - Product     | MVP scope, priority, trade-off decisions                        |
| Danny - PM/Pitch        | timeline, blockers, demo evidence, rehearsal needs              |

# 35. MVP build roadmap and weekly outcomes

The roadmap should align with the Hack Jam cadence. The MVP is not the full vision. It should prove the closed-loop concept: verified student marketplace, protected order state, fallback/request capture, demand signal and Incubation Hub actionability.

Every sprint should end with something visible: research evidence, a clickable prototype, a working feature, a preview link, or a test result.

| Week               | Developer/build output                                              |
|--------------------|---------------------------------------------------------------------|
| 2 - research       | technical spikes: identity, payments, data model, API map, risk log |
| 3 - requirements   | ERD, RBAC, OpenAPI draft, repo setup, issue backlog                 |
| 4 - design support | Figma handoff checks, component list, API mocks                     |
| 5 - MVP build      | auth stub, listings, search, request, order state, dashboards       |
| 6 - testing        | Playwright flows, accessibility, security checks, bug fixes         |
| 7 - hardening      | demo seed data, fallback demo path, final docs                      |
| 8 - Demo Day       | stable deployment, technical explanation, evidence package          |

## 36. MVP cutline from a technical perspective

From a build perspective, the MVP must remain small enough to test. The key is not the number of features; it is showing the system interaction: verified identity, marketplace search, transaction state, fallback request, demand signal, Incubation dashboard.

Several features should be mocked or scoped down for demo: real payment settlement, crypto, advanced reels, advanced courier dispatch, real ID verification and live campus operations.

| Must implement or simulate | Can stub/mock             | Later research                   |
|----------------------------|---------------------------|----------------------------------|
| auth/session + roles       | Eduvos SSO integration    | production identity agreements   |
| listings + search          | AI ranking model          | full recommender                 |
| order state machine        | live PSP money hold       | production payment orchestration |
| request + fallback         | cross-campus logistics    | full courier network             |
| seller dashboard           | advanced seller analytics | subscription billing             |
| Incubation dashboard       | AI-generated plan text    | fully automated interventions    |
| dispute/report prototype   | legal dispute workflow    | formal appeals infrastructure    |

## 37. Performance and scalability research

The MVP does not need enterprise scale, but it must not be architected in a way that blocks all-campus scale. The system should separate campus filters, category indexes, listing search, event feeds and analytics aggregation from heavy real-time or AI operations.

The largest performance risks are media-heavy reels, unbounded search queries, real-time chat growth, large file uploads, dashboard analytics and AI moderation calls. These should be throttled, paginated, cached and queued where appropriate.

| Risk                | Technical mitigation                                                |
|---------------------|---------------------------------------------------------------------|
| Large listing feed  | pagination, campus/category indexes, search service later           |
| Reels/video         | phase after core, compressed media, moderation queue                |
| Real-time chat      | thread limits, archive policies, careful subscriptions              |
| Analytics dashboard | pre-aggregate signals, do not query raw events live                 |
| AI calls            | queue, cache, confidence thresholds, fail closed for risky listings |
| File uploads        | size/type limits, virus scanning, private buckets                   |

## 38. Observability and analytics for developers

The team needs operational visibility, not just product analytics. Developers need logs, errors, audit events, transaction IDs and trace IDs. Every sensitive action should be traceable from UI to API to database/audit row.

Product analytics and security audit logs should not be mixed. Product analytics can use pseudonymous events. Audit logs need security-grade detail and restricted access.

| Telemetry type      | Examples                                             | Audience                              |
|---------------------|------------------------------------------------------|---------------------------------------|
| Product events      | search_performed, listing_viewed, request_created    | Research/Product/Incubation aggregate |
| Business events     | order_completed, payment_released, dispute_resolved  | operations/admin                      |
| Security audit      | role change, enforcement, evidence view              | authorized admins                     |
| Error logs          | API failure, validation failure, PSP webhook failure | developer/admin                       |
| Performance metrics | latency, build time, page loads                      | developer                             |

# 39. Design system and frontend component research

The frontend should use a small, consistent component system rather than custom components on every page. This matters because the marketplace has many modes; inconsistent UI will make the product feel cluttered and hard to trust.

Define design tokens for neutral colors, typography, spacing, status colors and trust badges. Use components for ListingCard, SellerBadge, TrustMeter, AvailabilityPill, OrderTimeline, PaymentProtectionBox, EvidenceUploader, DisputeStatus, SignalCard and DashboardMetric.

| Component            | States to design/test                                               |
|----------------------|---------------------------------------------------------------------|
| ListingCard          | available, out of stock, service slot, other campus, trusted seller |
| SearchFallback       | no local result, other campus, post request, similar category       |
| OrderTimeline        | requested, accepted, paid, ready, delivered, confirmed, disputed    |
| PaymentProtectionBox | deposit, held funds, release, partial refund                        |
| EvidenceUploader     | uploading, accepted, invalid type, private evidence                 |
| SignalCard           | low/medium/high confidence opportunity                              |
| BusinessHealthWidget | staff only, explainable indicators                                  |

## 40. Open research questions for technical spikes

The technical research should produce decisions, not just opinions. Each spike should have a question, method, owner and output. This keeps the team from arguing abstractly about technology choices.

A spike should be small: one day or less for most topics. The output can be a short ADR, prototype repo branch, diagram, pricing table or risk note.

| Spike         | Question                                                     | Output                                 |
|---------------|--------------------------------------------------------------|----------------------------------------|
| Identity      | Can Eduvos provide SSO/student verification route?           | identity options ADR                   |
| PSP           | Which provider can support marketplace holds/splits/refunds? | payment architecture note              |
| RLS           | Can Supabase policies enforce core objects safely?           | policy test matrix                     |
| PWA           | Can desktop/mobile install experience meet expectations?     | PWA proof + limitations                |
| File security | How to protect evidence/digital files?                       | bucket policy design                   |
| AI moderation | Can AI reliably pre-check listings?                          | prompt/rules prototype + failure cases |
| Reels         | Can video feed exist without derailing MVP?                  | defer/phase decision                   |

# 41. Architecture Decision Records

Every major technical decision should be recorded as an Architecture Decision Record. This is useful for team memory, judging clarity and handover to Eduvos. ADRs should be short: context, decision, consequences and alternatives considered.

The first ADRs should cover delivery surface, data platform, auth strategy, payments strategy, API contract approach, RLS/authorization approach, media storage, and AI moderation policy.

| ADR     | Decision to document                             |
|---------|--------------------------------------------------|
| ADR-001 | PWA-first MVP vs native-first                    |
| ADR-002 | Supabase/Postgres vs alternative backend         |
| ADR-003 | Modular monolith vs microservices                |
| ADR-004 | PaymentHold state with PSP as system of record   |
| ADR-005 | Role + object authorization model                |
| ADR-006 | AI as assistant with human review for punishment |
| ADR-007 | Demand Intelligence event taxonomy               |
| ADR-008 | Private evidence storage and audit logging       |

## 42. Developer deliverables checklist

The developer workstream should deliver research artifacts before code, then code artifacts before the final pitch. This checklist keeps the build grounded and prevents uncontrolled scope growth.

| Deliverable               | Definition of done                                       |
|---------------------------|----------------------------------------------------------|
| Tech-stack recommendation | options compared, risks stated, final choice provisional |
| Architecture diagram      | modules, boundaries, external systems, data flows        |
| ERD v0                    | entities, relationships, sensitive data markings         |
| RBAC matrix               | roles, object permissions, sensitive actions             |
| OpenAPI draft             | core endpoints, request/response patterns, errors        |
| GitHub repo               | structure, issues, PR template, CI workflow              |
| Wireframes support        | API/data notes attached to each P0 flow                  |
| Testing matrix            | unit/component/E2E/security/accessibility plan           |
| Demo build                | seed data, stable preview URL, fallback plan             |

## 43. Developer risk register

The risk register should stay visible in GitHub/Miro. Each risk must have an owner, mitigation and next decision point. Do not bury payment/legal/privacy risks until the end of the Hack Jam.

| Risk                                     | Impact                    | Mitigation                                   |
|------------------------------------------|---------------------------|----------------------------------------------|
| Payment hold not supported by chosen PSP | core trust model breaks   | research PSP first; mock for MVP             |
| 2% commission cannot cover fees          | business model inaccurate | model processor fees separately              |
| RLS/authorization gaps                   | data breach/fraud         | server checks + RLS + tests                  |
| Scope overload                           | unfinished MVP            | strict P0 outline                            |
| AI false positives                       | unfair blocking           | human review + appeals                       |
| Food/courier liability                   | institutional risk        | treat as research/phase with policy sign-off |
| Reels moderation burden                  | unsafe content/clutter    | phase and moderate strongly                  |
| Live location privacy                    | student safety issue      | optional, purpose-limited, short retention   |

## 44. Suggested starter repo files

The following file plan is a research recommendation for the first real repository. It gives the team a starting point without forcing a specific provider before technical spikes finish.

| File/folder                                      | Purpose                                          |
|--------------------------------------------------|--------------------------------------------------|
| README.md                                        | project thesis, setup, commands, links           |
| CONTRIBUTING.md                                  | Git workflow, PR rules, branch naming            |
| SECURITY.md                                      | reporting, secrets, security expectations        |
| docs/architecture/ADR-000-template.md            | architecture decisions                           |
| docs/api/openapi.yaml                            | API contract draft                               |
| docs/security/threat-model.md                    | abuse paths and mitigations                      |
| apps/web/src/app                                 | Next.js app routes                               |
| apps/web/src/modules                             | marketplace, orders, trust, payments, incubation |
| packages/shared/src                              | types, enums, validators                         |
| packages/db/schema.prisma or supabase/migrations | database source of truth                         |
| tests/e2e                                        | Playwright user journeys                         |

# 45. Sample package scripts and CI gates

The exact commands depend on the final stack, but the principle is stable: a change cannot merge unless it formats, lints, type-checks, tests and builds. E2E tests can start with the highest-value flow and expand over time.

| Script     | Purpose                |
|------------|------------------------|
| dev        | run local app          |
| lint       | static lint checks     |
| typecheck  | TypeScript correctness |
| test       | unit tests             |
| test:e2e   | critical user journeys |
| build      | production build       |
| db:migrate | apply migrations       |
| db:seed    | seed demo/test data    |
| storybook  | component docs/testing |
| format     | format code            |

## 46. Wireframe/API traceability map

Wireframes are not just pictures. Every screen implies data, API calls, error states, permissions and analytics events. The team should attach technical notes to each Figma flow before development starts.

| Screen/flow          | Primary APIs                                  | Events                              |
|----------------------|-----------------------------------------------|-------------------------------------|
| Marketplace home     | GET /listings, GET /events, GET /signals      | search_performed, listing_viewed    |
| Search fallback      | GET /listings?campus=all, POST /requests      | search_zero_result, request_created |
| Seller onboarding    | POST /identity/id-check, POST /seller-profile | seller_onboard_started/completed    |
| Listing creation     | POST /listings, POST /ai/check-listing        | listing_submitted, listing_flagged  |
| Checkout             | POST /orders, POST /payments/hold             | order_started, payment_protected    |
| Order tracking       | GET /orders/{id}, PATCH /orders/{id}/state    | order_state_changed                 |
| Dispute              | POST /disputes, POST /evidence                | dispute_opened, evidence_uploaded   |
| Incubation dashboard | GET /signals, GET /business-health            | signal_viewed, intervention_created |

## 47. Build-readiness scorecard

Before moving from research into implementation, use a simple readiness scorecard. If a module scores low, it can still be prototyped visually, but the team should avoid pretending it is production-ready.

| Module                  | Build readiness | Reason                                  |
|-------------------------|-----------------|-----------------------------------------|
| Listings/search         | High            | standard marketplace patterns           |
| Seller onboarding       | Medium          | depends on verification path            |
| Protected payment state | Medium          | UX can be built; provider/legal unknown |
| Real payment release    | Low             | compliance and PSP dependency           |
| Requests/demand signals | Medium-high     | data pipeline is feasible               |
| Student courier network | Low-medium      | operations and liability unknown        |
| AI listing checks       | Medium          | useful with human review                |
| Incubation Health Score | Medium          | requires co-design and explainability   |
| Reels/social feed       | Low-medium      | moderation and scope risk               |

## 48. Recommended focus for the developer build

The developer workstream should not try to build the entire researched ecosystem in one pass. It should build a credible end-to-end proof of the unique loop. The demo should show a student searching, failing locally, seeing fallback options, creating a request, generating a demand signal, and showing how the Incubation Hub can act on it.

This proves the concept better than building shallow versions of reels, crypto, subscriptions, rentals, events and couriers all at once. The other modes should remain documented as phased research.

| Build priority                     | Why                                |
|------------------------------------|------------------------------------|
| Verified user/session + roles      | trust boundary for all flows       |
| Campus-first search + availability | core student value                 |
| Listing + request flow             | basic marketplace + demand capture |
| Order state + protected payment UX | trust mechanism without live funds |
| Demand signal dashboard            | unique differentiator              |
| Incubation intervention prototype  | ties marketplace to Eduvos purpose |
| Dispute/report basic flow          | safety credibility                 |
| Testing/demo seed data             | reliable presentation              |

## 49. References

References are current research sources used to shape technical recommendations. Re-verify versions, pricing and plan limits immediately before final implementation.

| ID  | Source                                                                                                                                                                                                                                                                                                                                                                               |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| D1  | MDN - Making PWAs installable: manifest, HTTPS and platform support.<br><a href="https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable">https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable</a>                                                                                              |
| D2  | Expo EAS Build docs - hosted builds for React Native/Expo binaries. <a href="https://docs.expo.dev/build">https://docs.expo.dev/build</a>                                                                                                                                                                                                                                            |
| D3  | Next.js Route Handlers docs - custom request handlers in app directory.<br><a href="https://nextjs.org/docs/app/getting-started/route-handlers">https://nextjs.org/docs/app/getting-started/route-handlers</a>                                                                                                                                                                       |
| D4  | Supabase Docs - Postgres, Auth, Storage, Realtime and Edge Functions. <a href="https://supabase.com/docs">https://supabase.com/docs</a>                                                                                                                                                                                                                                              |
| D5  | GitHub Docs - protected branches and review/status-check requirements. <a href="https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches">https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches</a> |
| D6  | GitHub Actions workflow syntax. <a href="https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax">https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax</a>                                                                                                                                                                  |
| D7  | Vercel environments - local, preview, production. <a href="https://vercel.com/docs/deployments/environments">https://vercel.com/docs/deployments/environments</a>                                                                                                                                                                                                                    |
| D8  | Vercel environment variables. <a href="https://vercel.com/docs/environment-variables">https://vercel.com/docs/environment-variables</a>                                                                                                                                                                                                                                              |
| D9  | OWASP API Security Top 10 2023 - Broken Object Level Authorization.<br><a href="https://api-security.owasp.org/editions/2023/en/0x11-t10/">https://api-security.owasp.org/editions/2023/en/0x11-t10/</a>                                                                                                                                                                             |
| D10 | OpenAPI Specification 3.1.0. <a href="https://spec.openapis.org/oas/v3.1.0">https://spec.openapis.org/oas/v3.1.0</a>                                                                                                                                                                                                                                                                 |
| D11 | Supabase Storage access control and RLS. <a href="https://supabase.com/docs/guides/storage/security/access-control">https://supabase.com/docs/guides/storage/security/access-control</a>                                                                                                                                                                                             |
| D12 | Supabase Realtime database changes. <a href="https://supabase.com/docs/guides/realtime/subscribing-to-database-changes">https://supabase.com/docs/guides/realtime/subscribing-to-database-changes</a>                                                                                                                                                                                |
| D13 | Playwright docs - E2E testing across browsers. <a href="https://playwright.dev/">https://playwright.dev/</a>                                                                                                                                                                                                                                                                         |
| D14 | Storybook docs - UI component testing and visual tests. <a href="https://storybook.js.org/docs/writing-tests">https://storybook.js.org/docs/writing-tests</a>                                                                                                                                                                                                                        |
| D15 | OWASP API Security Project overview. <a href="https://owasp.org/projects/api-security-project">https://owasp.org/projects/api-security-project</a>                                                                                                                                                                                                                                   |
| D16 | Supabase Row Level Security docs. <a href="https://supabase.com/docs/guides/database/postgres/row-level-security">https://supabase.com/docs/guides/database/postgres/row-level-security</a>                                                                                                                                                                                          |

# Appendix A. Starter repository skeleton

**Repository principle: one clear source of truth for code, docs, API contracts, database schema and tests.**

Use this as a starter blueprint when the GitHub repository is created. It is intentionally simple and can evolve after technical spikes.

| Folder                           | Contents                               |
|----------------------------------|----------------------------------------|
| apps/web                         | Next.js app, routes, components, pages |
| apps/web/src/modules/marketplace | listing/search/request domain          |
| apps/web/src/modules/orders      | order state, checkout, tracking        |
| apps/web/src/modules/trust       | reviews, disputes, reports             |
| apps/web/src/modules/incubation  | signals, dashboards, interventions     |
| packages/shared                  | types, enums, zod schemas, constants   |
| packages/db                      | schema, migrations, seed scripts       |
| docs/api                         | OpenAPI, endpoint notes                |
| docs/architecture                | ADRs, ERD, RBAC, diagrams              |
| tests/e2e                        | Playwright flows                       |

## Appendix B. Sample issue template

Use an issue template that forces evidence, ownership and acceptance criteria before build work begins.

```
--- title: "[Feature] Short name" labels: feature, needs-scope --- Problem / evidence: User role:  
Requirement ID: User story: Acceptance criteria: Figma / wireframe link: API/data impact:  
Analytics event impact: Security/privacy notes: Test plan: Owner: Priority: P0 / P1 / P2
```

# Appendix C. Sample PR template

A PR should make it easy to review not only code quality but research and product alignment.

Linked issue: Summary: What changed: Screenshots / preview URL: Tests run: Security/privacy checks: Database/API changes: Known risks: Rollback plan: Reviewer checklist: - matches acceptance criteria - no direct main push - no new secrets - role/object authorization checked - empty/error/loading states handled

# Appendix D. Critical E2E test list

These tests should be created before Demo Day. They are more important than testing every visual variation.

| Test                                            | Goal                        |
|-------------------------------------------------|-----------------------------|
| buyer_can_find_local_listing                    | basic discovery works       |
| buyer_can_fallback_to_other_campus              | never lose client principle |
| buyer_can_create_request_after_zero_result      | demand capture              |
| seller_can_create_listing_with_compliance_check | AI/policy gate              |
| checkout_shows_payment_protection               | trust UX                    |
| seller_cannot_edit_other_seller_listing         | object auth                 |
| buyer_cannot_view_unrelated_dispute_evidence    | privacy                     |
| moderator_can_block_listing_with_reason         | enforcement path            |
| incubation_staff_can_view_aggregate_signal      | Hub intelligence            |
| anonymous_user_cannot_access_private_api        | auth baseline               |

# Appendix E. Final technical thesis

The developer build should prove that EduHackers can become an Eduvos student marketplace operating system, not merely a digital notice board. The technical strategy is to keep the first build narrow, traceable and secure while preserving a model that can expand into mobile, events, rentals, couriers, AI support and Incubation Hub interventions.

**If the team can demonstrate verified users, campus-first discovery, protected order state, fallback capture, demand intelligence and Incubation action in one reliable demo, it will have a stronger technical story than a broader but shallow feature showcase.**